

实验概述

1. 实验目的

- （1）理解流水线CPU的结构及工作原理；
- （2）理解流水线冲突产生的原因及其解决方法；
- （3）掌握流水线CPU的设计与实现方法。

2. 实验内容

本实验要求改造实验1所设计的完整单周期CPU，从而实现支持miniRV或miniLA指令集的流水线CPU。

具体要求如下：

- （1）至少实现 **五级** 流水线；
- （2）支持miniRV或miniLA的全部指令；
- （3）改造完整单周期CPU的数据通路，设计理想流水线CPU；
- （4）实现静态分支预测，解决控制冒险；
- （5）实现流水线暂停及数据前递机制，解决数据冒险；
- （6）通过调试，使流水线CPU通过Basic Trace测试；
- （7）与组员讨论协作，把流水线CPU集成到SoC，并使用AXI Trace调试和验证流水线SoC；
- （8）与组员讨论协作，在SoC上完成CoreMark或LLAMA2推理程序的下板测试；
- （9）尝试优化性能，减少CoreMark的运行时间，或提升LLAMA2的推理速率；
- （10）模板工程默认给SoC提供50MHz的时钟，可根据实际情况自行调节频率（调节方法见[实验1-实验原理-功能部件设计-1.1节的图3-3](#)），要求单周期SoC频率 **不低于25MHz**、流水线SoC频率 **不低于50MHz**。
- （11）下板检查时，要求不能有时序违例，如图0-1所示。

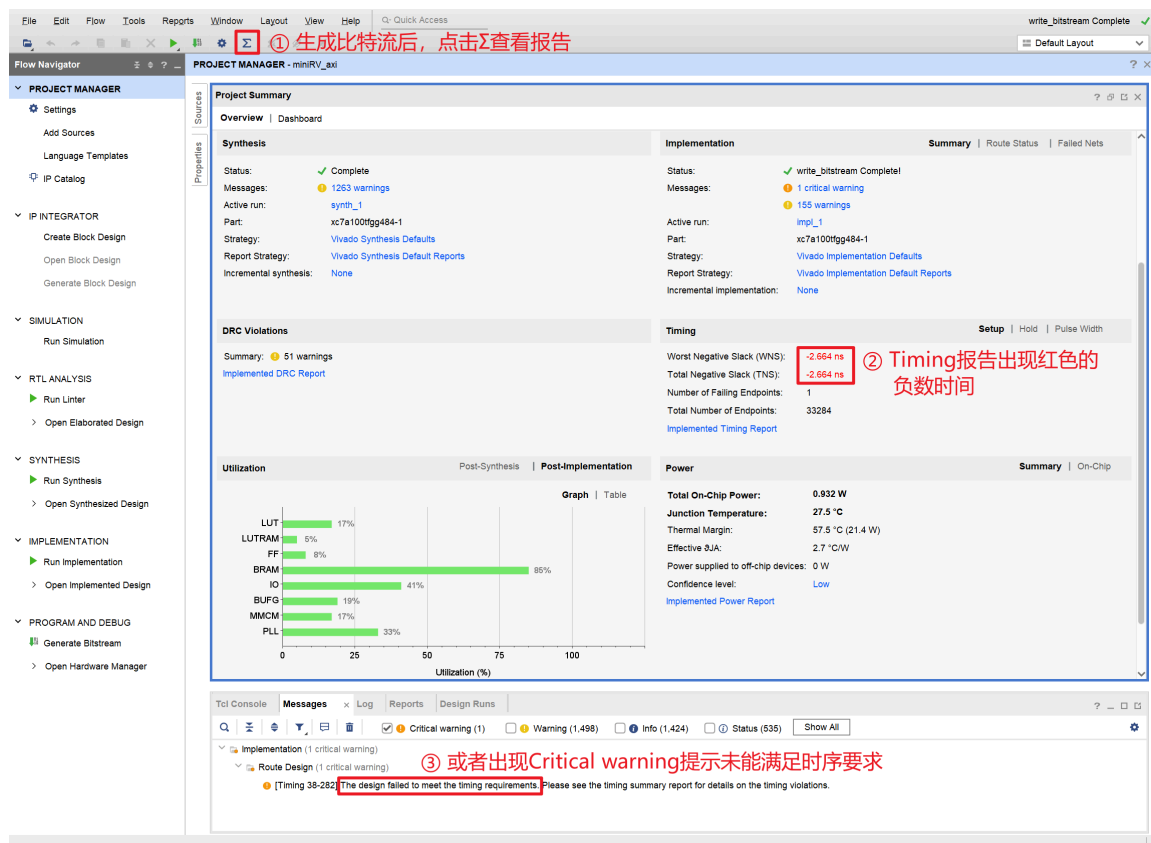


图0-1 生成比特流后，出现时序违例的示意图

流水线概述

流水线 (Pipeline) 技术是指在程序执行时, 将多条指令的执行操作在时间上进行重叠, 从而提高指令级并行度 (Instruction-Level Parallelism, ILP) 的一种技术手段。IBM在1961年推出的IBM 7030 Stretch超级计算机是世界上最先使用流水线技术的计算机。流水线技术现已广泛应用于现代处理器当中。

流水线技术的基本原理是, 在CPU中利用多个不同功能的电路单元组成一条指令处理流水线, 然后将一条指令的执行过程切分成多个阶段, 并由这些电路单元分别依次执行, 一方面能够实现在一个时钟周期内同时执行多条指令, 提高各电路单元的利用率, 另一方面有利于提高CPU主频, 从而提高CPU的运算速度。

令 $T_{\text{non-Pipeline}}$ 表示程序的非流水式执行时间, T_{Pipeline} 表示程序的流水式执行时间, N 表示流水线级数, 则在理想情况下, 流水线处理器的执行时间可用以下公式计算:

$$T_{\text{Pipeline}} = \frac{T_{\text{non-Pipeline}}}{N}$$

1. 流水线划分

1.1 流水级的划分原则

在设计流水线处理器时, 流水级 (Pipeline Stage) 的划分直接决定了流水线是否高效。在划分流水线时, 通常需要从 **功能**、**延时**、**位宽** 三个方面进行考虑。

1.1.1 按功能划分

如果电路本身具有若干个明确的子功能, 或其工作时存在若干个逻辑上具备先后关系的阶段, 则可以将各项子功能或各个阶段分别划分成一个流水级。

举个栗子 🌰

例如, CPU执行指令的过程可以分成取指、译码、执行、访存、写回五个阶段, 因此常常会将流水线划分成对应的五个流水级。

1.1.2 按延时分

单周期CPU的主频取决于执行时间最长的指令 (如Load指令)。同理, 流水线CPU的主频取决于耗时最长的流水级。因此, 在设计流水线时, 应尽量使得每个流水级的延时相近, 因为这样才能使处理器运行在更高的频率, 从而获得更好的性能。

1.1.3 按位宽划分

划分流水线时，需要增加流水线寄存器。为了节省寄存器的数量和面积，在设计流水线时，还需尽量使得各流水级之间的交互信号尽可能少。

 补充说明

实际划分流水线时，一般需要同时考虑以上3项的流水线划分原则。

1.2 流水线实例

1.2.1 RISC经典五级流水线

按照RISC处理器的经典五级流水结构，可将单周期CPU的执行阶段划分为如图1-1所示的五级流水线。

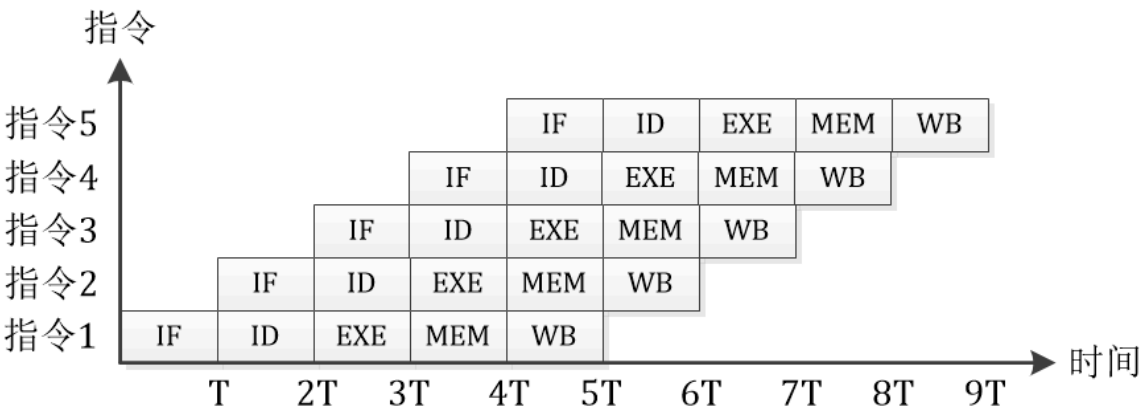


图1-1 五级流水线

- **取指** 阶段（Instruction Fetch，**IF**）：是指将指令从指令存储器中读取出来的过程。
- **译码** 阶段（Instruction Decode，**ID**）：是指将取指阶段取出的指令进行翻译的过程。译码阶段将得到指令的操作码、功能码、操作数寄存器号等信息，然后从寄存器堆（Register File）取出操作数，同时产生指令执行所需的控制信号。
- **执行** 阶段（instruction EXecute，**EX**）：是指根据指令的操作码和功能码，对指令操作数进行运算的过程。如果指令是一条加法运算指令，则对操作数进行加法操作；如果是减法运算指令，则进行减法操作。CPU中的算术逻辑单元（Arithmetic Logical Unit，ALU）是实施具体运算的硬件功能单元，是执行阶段的核心部件。
- **访存** 阶段（MEMory access，**MEM**）：是指访存指令从存储器读出数据，或将数据写入存储器的过程。
- **写回** 阶段（Write Back，**WB**）：是指将指令执行结果写回到目标寄存器的过程。运算类指令的执行结果是执行阶段的输出，而访存指令的执行结果则是访存阶段从存储器读取出的数据。

1.2.2 ARM10六级流水线

ARM10处理器将指令的执行过程划分成取指、分派、译码、执行、访存、写回六级流水线。

- 取指阶段（IF）：进行指令分支预测及读取指令。
- 分派阶段（instruction ISSue, ISS）：判断指令是否是协处理器指令。
- 译码阶段（ID）：根据指令格式解析指令，并从寄存器堆中取出源操作数。
- 执行阶段（EX）：使用ALU完成指令所需的运算操作。
- 访存阶段（MEM）：进行指令的访问数据存储器操作。
- 写回阶段（WB）：将指令执行结果写回到寄存器堆中的指定寄存器。

1.2.3 ARM11八级流水线

ARM11处理器将指令的执行过程划分成八级流水线。流水线的前端包括取指1、取指2、译码和分派四个阶段；流水线后端则分成了ALU、MAC（Multiply-ACcumulate，乘累加）和访存三条不同的子流水线，如图1-2所示。

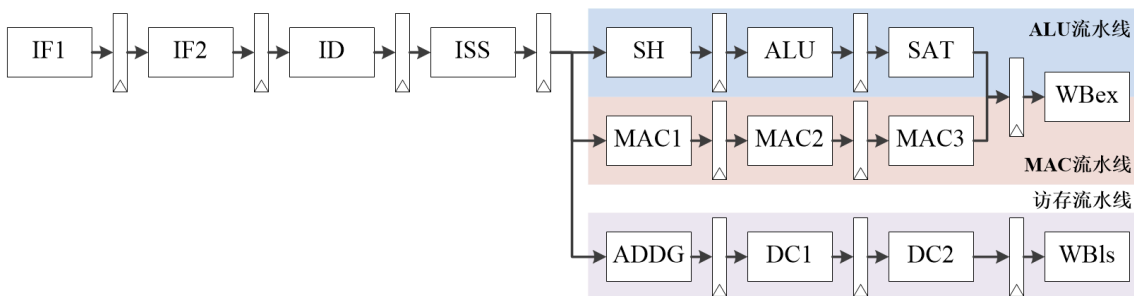


图1-2 ARM11八级流水线结构

- 取指阶段1（IF1）：进行动态分支预测及读取指令。
- 取指阶段2（IF2）：进行静态分支预测。
- 译码阶段（ID）：根据指令格式解析指令，并判断指令是否是协处理器指令。
- 分派阶段（ISS）：从寄存器堆中取出源操作数，并分派指令到相应的子流水线。
- 移位阶段（SHift, SH）：进行数据移位操作。
- 运算阶段（ALU）：进行整数的算术逻辑运算。
- 饱和阶段（SATuration, SAT）：对整数进行限幅操作。
- 执行写回阶段（Write Back execution, WBex）：乘法或主执行流水线（Main Execution Pipeline）的写回阶段——将运算结果写回到寄存器堆的指定寄存器。
- 乘累加阶段1（MAC1）：乘累加运算的第1个阶段。
- 乘累加阶段2（MAC2）：乘累加运算的第2个阶段。
- 乘累加阶段3（MAC3）：乘累加运算的第3个阶段。

- 地址生成阶段 (ADDRESS Generation, ADDG): 通过计算产生访存操作地址。
- 数据缓存读取阶段1 (Data Cache access, DC1): 访问数据Cache的第1个阶段。
- 数据缓存读取阶段2 (Data Cache access, DC2): 访问数据Cache的第2个阶段。
- 访存写回阶段 (Write Back from Load Store Unit, WBIs): 访存结果的写回——将加载存储单元 (Load Store Unit, LSU) 的数据写回到寄存器堆的指定寄存器。

2. 流水寄存器

理想情况下,流水线的各个阶段总是在执行不同的指令。为了保证指令获得正确的执行结果,需要在相邻的两个执行阶段之间添加寄存器,用于暂存指令执行的中间结果,这些寄存器被称为流水线的段寄存器(或流水寄存器),如图1-3所示。

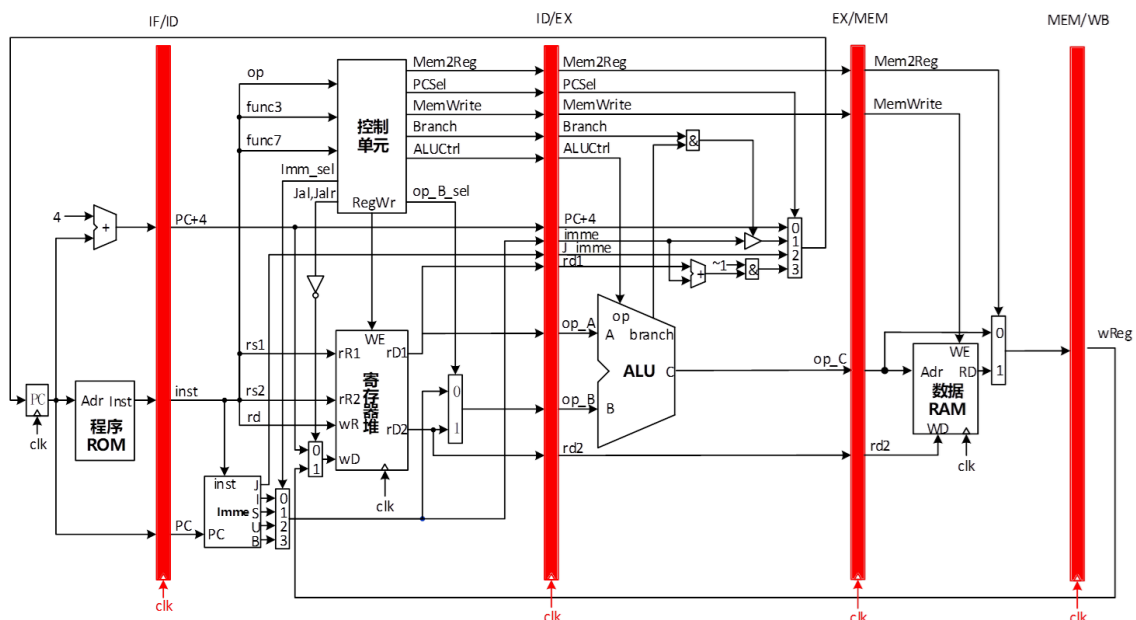


图1-3 流水线的段寄存器

流水线寄存器的主要作用是：

- (1) 暂存流水级的输出信息,交给下一级处理,并共享给其他指令;
- (2) 切割单周期处理器的组合逻辑,降低指令的执行时延,提升CPU时钟频率。

下面以IF-ID-EX-MEM-WB的五级流水结构为例,对段寄存器的设计进行简要介绍。

• IF/ID流水寄存器

取指级和译码级之间的段寄存器用于传递取出的指令及PC值等,如图1-4所示。



图1-4 IF/ID流水寄存器

IF/ID流水寄存器的参考RTL实现如下列代码所示。

```
1  always @ (posedge clk or posedge rst) begin
2      if (rst) id_pc <= 32'h0;
3      else     id_pc <= if_pc;
4  end
5
6  always @ (posedge clk or posedge rst) begin
7      if (rst) id_inst <= 32'h0;
8      else     id_inst <= if_inst;
9  end
```

小提示

流水线的段寄存器是一个概念上的寄存器，其实现形式不定 —— 可以仅用一个若干位的寄存器实现，也可以用若干个“零散的、较小的”寄存器凑在一起形成一个“完整的”段寄存器。

• ID/EX流水寄存器

译码级和执行级之间的段寄存器用于传递当前指令的操作类型、立即数、源操作数及各控制信号，如图1-5所示。



图1-5 ID/EX流水寄存器

- **EX/MEM流水寄存器**

执行级和访存级之间的段寄存器用于传递当前指令的访存相关控制信号以及ALU计算结果等，如图1-6所示。

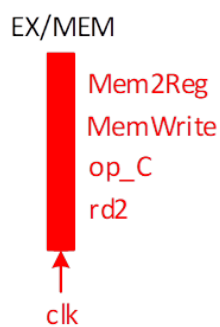


图1-6 EX/MEM流水寄存器

- **MEM/WB流水寄存器**

访存级和写回级之间的段寄存器用于传递当前指令的写回结果，如图1-7所示。



图1-7 MEM/WB流水寄存器

 请注意

上述流水线寄存器仅为示例，同学们需根据自己的设计来决定流水线的级数和段寄存器的结构、存储的信息等。

理想流水线

理想流水线一般指流水线中不存在冲突或冒险（Hazzard）问题、不存在资源冲突，且进入流水线的指令相互之间不存在依赖关系或相关关系。

通过修改单周期CPU的数据通路，可以实现简易流水线CPU的设计。

1. 修改取指逻辑

在理想流水线中，每个流水线阶段都有一条指令正在执行，即流水线的时空图是稠密的。以“IF-ID-EX-MEM-WB”的RISC架构经典五级流水线为例，其理想流水线如图2-1所示。

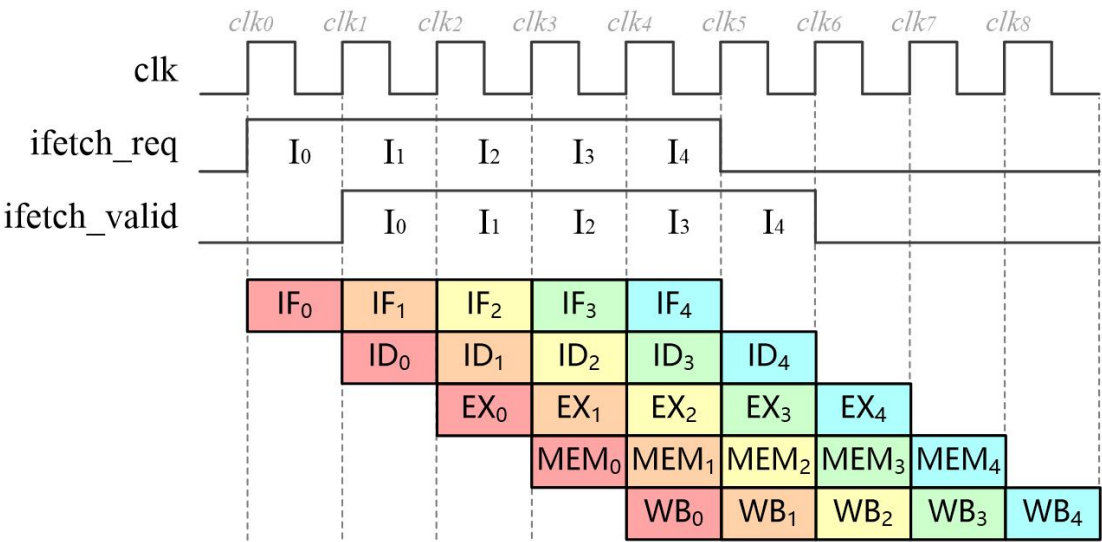


图2-1 理想流水线的执行规律示意图

观察图2-1可知，要想达到理想流水线的效果，必须在取到一条指令时（即 ifetch_valid 有效时），立即发出下一条指令的取指请求。比如，当指令 I₀ 在 clk₁ 取到机器码时，必须同时发出指令 I₁ 的取指请求。因此，我们需要把单周期CPU的取指逻辑修改成：

```
cpu_core.v
85 // 复位信号发生边沿变化时首次取指；上一条已取回，同时立即取下一条
86 assign ifetch_req = first_req | ifetch_valid;
87 assign ifetch_addr = pc;
```

？ 指令什么时候在IF阶段？ 什么时候进入ID阶段？

CPU发出某条指令的取指请求时，这条指令就处于IF阶段。

例如，在图2-1的 clk_0 处，`ifetch_req` 有效，假设此时 `ifetch_addr` 等于指令 I_0 的地址，则 I_0 处于IF阶段。

在CPU发出取指请求的下一拍开始，当指令机器码被取回到CPU时，指令就处于ID阶段。

例如，在图2-1中，指令 I_0 在 clk_0 取指，而 `ifetch_valid` 在下一拍 clk_1 有效，因此 I_0 在 clk_1 进入ID阶段。

2. 单周期数据通路改造

🔥 对于具有一定复杂度的系统，先实现大体架构，然后再逐步考虑各种特殊情况

理想流水线是功能最简单的流水线，不能停顿，不能处理指令相关。

实现理想流水线时，暂时不要考虑分支跳转、访存、乘除法、数据相关等情况。

以下以图2-2所示的单周期CPU为例，介绍如何将单周期CPU改造成“取指-译码-执行”的三级理想流水CPU。

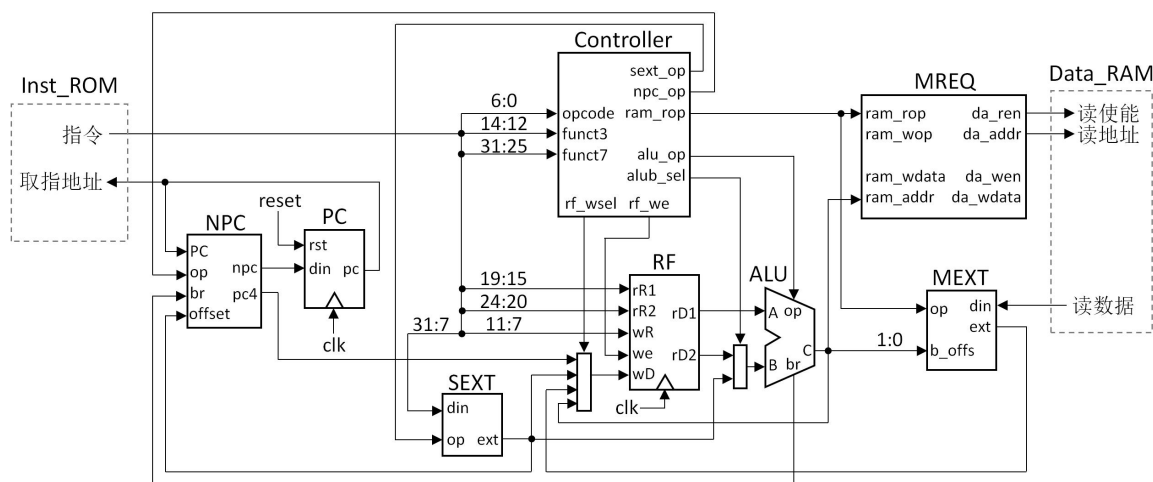


图2-2 单周期CPU数据通路示例

第一步，根据目标流水线的阶段构成，分割单周期CPU的数据通路，并在分割处加入流水寄存器，将各阶段之间的接口信号缓存起来，如图2-3所示。

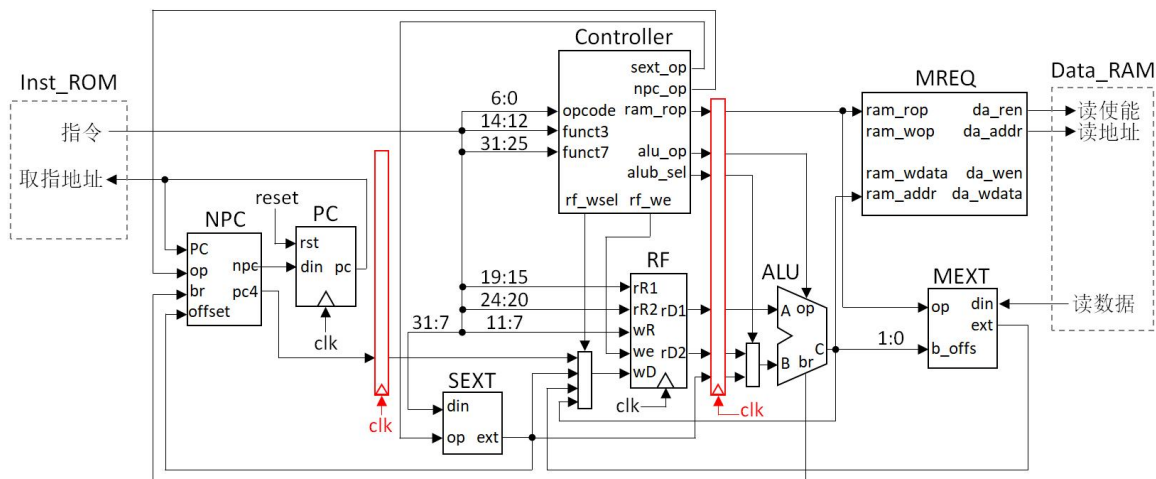


图2-3 分割单周期CPU数据通路

第二步，修改写回逻辑，确保指令的执行结果能够写回到正确的寄存器。将原有的单周期数据通路“切割”成三个流水级之后，每个流水级都有一条指令在执行。假设在某个时刻，指令 I_0 、 I_1 、 I_2 分别处于执行阶段、译码阶段和取指阶段。当执行阶段的指令 I_0 需要写回时，寄存器堆的写使能信号、目标寄存器号和写数据信号都是由指令 I_1 译码产生的。此时，若进行写回操作，则将发生指令 I_0 的执行结果写回到指令 I_1 的目标寄存器的错误。为了避免此错误发生，需要在指令 I_0 从译码阶段进入到执行阶段时，将其写回的相关信号通过流水寄存器传递到执行阶段，如图2-4所示。

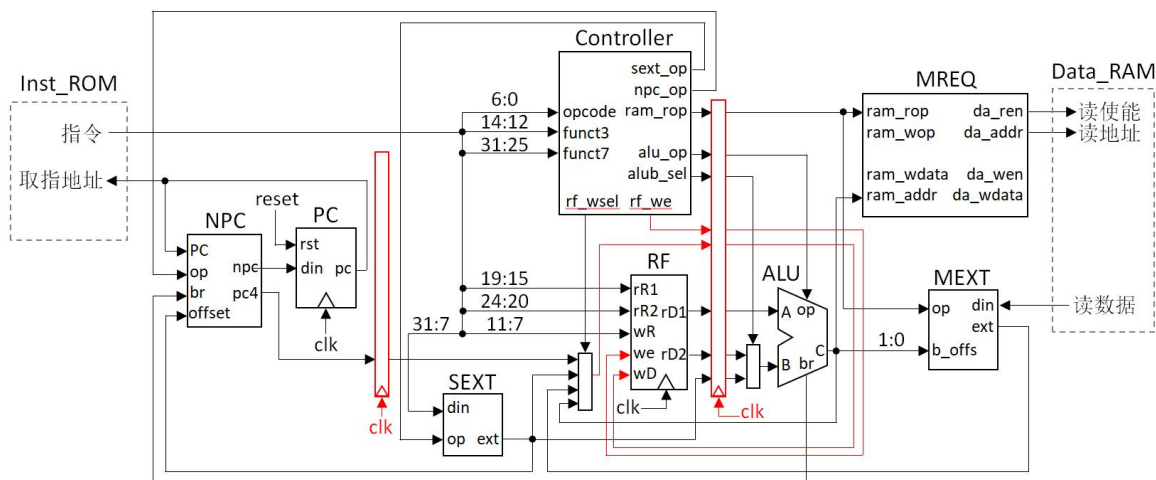


图2-4 修改写回逻辑

3. 理想流水线CPU的仿真测试

设计出理想流水线之后，可通过无指令相关性、无访存、无乘除法的简单程序进行仿真测试，从而初步验证流水线架构的正确性。

下面给出理想流水线的参考测试程序，同学们可根据实际情况修改其中的指令。


```

1 xori    t0, zero, 1
2 ori     t1, zero, 2
3 addi    t2, zero, 3
4 lui     t4, 0x4
5 auipc   t5, 0x0
6 slti    t6, zero, 2
7 add     s0, t0, t1

```

把上述程序保存成.asm文件，在RARS中进行汇编，如图2-5所示。

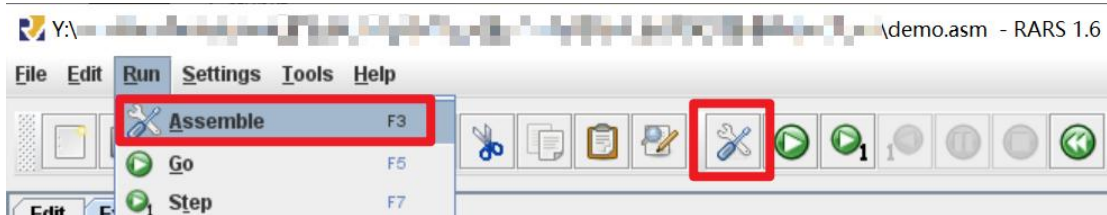


图2-5 在RARS中汇编代码

依次点击“File” -> “Dump Memory...”，选择导出方式为16进制文本格式，如图2-6所示。

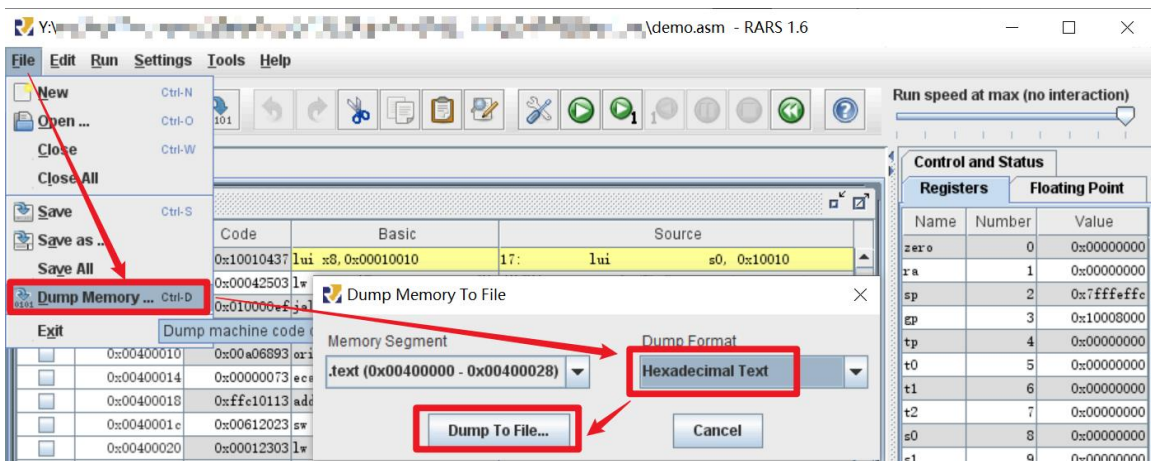


图2-6 在RARS中汇编代码

打开导出的文本文件，复制文件内的指令机器码，参考[实验1-实验原理-功能部件设计-3.2 Block Memory的初始化](#)生成.coe文件并将其导入工程的IROM IP核当中，即可运行功能仿真对理想流水线CPU进行调试和验证。

miniLA 

```

1 xori    t0, zero, 1
2 ori     t1, zero, 2
3 addi.w  t2, zero, 3
4 lui12i.w t4, 0x4
5 pcaddu12i t5, 0x0
6 slti    t6, zero, 2
7 add.w   s0, t0, t1

```

复制上述代码到LARS工具，依次点击汇编按钮和导出COE按钮，如图2-5A所示。



图2-5A 在RARS中汇编代码

参考实验1-实验原理-功能部件设计-3.2 Block Memory的初始化-图3-8，把下载的.coe文件导入工程的IROM IP核当中，即可运行功能仿真对理想流水线CPU进行调试和验证。

指令相关与流水线冒险

理想情况下，每个时钟周期都有一条指令流入流水线，并且指令装入完成后，每个时钟周期都有一条执行完成的指令从流水线中流出。实际上，指令之间可能存在依赖关系，导致指令流中的指令不能在期望的时钟周期进入流水线执行。

1. 指令相关

相关（Dependence）指的是指令之间存在的依赖关系。当两条指令之间存在相关关系时，它们就不能在流水线中重叠执行，或只能部分重叠执行。

指令之间的相关关系包括数据相关、名相关和控制相关三种。

- **数据相关**

数据相关又称真数据相关，指的是指令之间存在数据依赖关系。比如指令流中，指令 I_{k+1} 需要使用上一条指令 I_k 的执行结果。

- **名相关**

名相关指的是存在多条指令同时使用了同一个寄存器或存储单元，但这些指令之间不存在数据相关。

名相关包括反相关和输出相关两种。反相关指的是指令 I_{k+1} 的目标寄存器与指令流中的上一条指令 I_k 的源寄存器相同；输出相关则指的是指令 I_{k+1} 和指令 I_k 的目标寄存器相同。

名相关可通过寄存器重命名（Register Renaming）解决。寄存器重命名既可以使用编译器或汇编器静态实现，也可以使用硬件动态实现。

- **控制相关**

控制相关指的是分支指令引起的指令相关 —— 在指令流中，排在分支指令之后的指令是否执行，取决于分支指令的执行结果。

2. 流水线冒险

冒险（Hazard）指的是因为指令存在相关性而引起的流水线工作异常的现象。流水线出现冒险时，本来应该流入流水线的指令不能正常流入，使得指令之间的重叠执行程度降低，从而造成流水线性能的浪费。

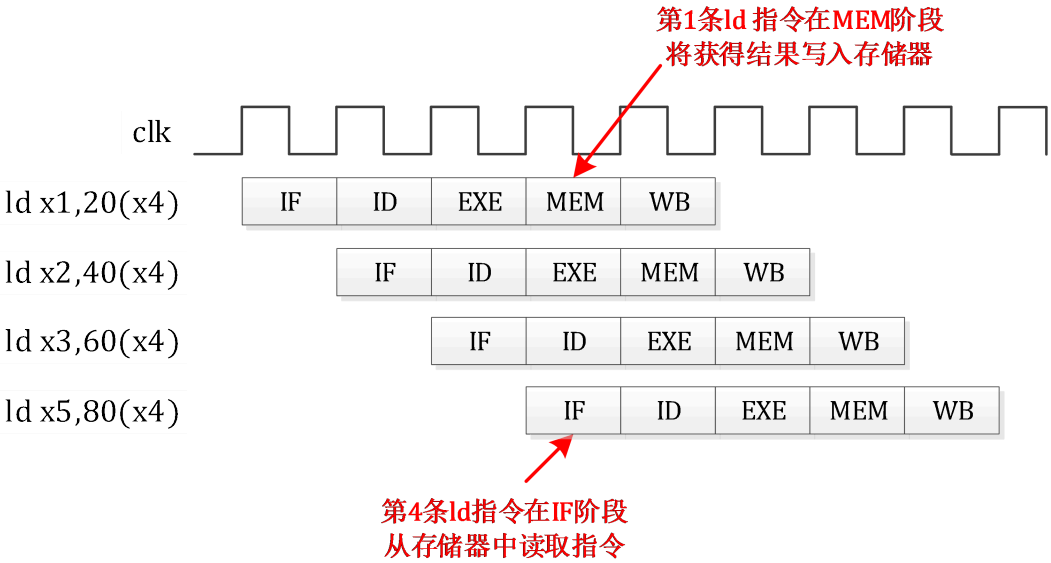
流水线冒险包括结构冒险、数据冒险和控制冒险三种。

2.1 结构冒险

当流水线的硬件资源满足不了指令重叠执行的需求时，指令无法在期望的时钟周期内完成执行。此种情形称为流水线的结构冒险。

**举栗说明**

假设某流水线处理器只有一个指令和数据共享的存储器，则访存阶段和取指阶段将同时访问存储器。若该存储器只有一个访问端口，则流水线将发生结构冒险，如图3-1所示。



第1条ld指令在MEM阶段
将获得结果写入存储器

第4条ld指令在IF阶段
从存储器中读取指令

图3-1 结构冒险示例

由单存储器引起的结构冒险可通过哈佛结构解决 —— 分别设置独立的指令存储器和数据存储器。

结构冒险会造成流水线停顿，从而影响其性能。如果结构冒险出现的频次较高，则可以通过增加硬件资源的方式解决；否则可以忽视之，从而降低成本。

2.2 数据冒险

存在数据相关的指令同时在流水线中执行时，重叠执行可能会破坏指令之间的数据依赖关系，从而导致指令执行结果出错。此种情形称为流水线的数据冒险。

数据冒险包括写后读（RAW）、读后写（WAR）、写后写（WAW）三种类型。

• RAW 冒险

指令 I_{k+1} 需要使用前一条指令 I_k 的计算结果。当这两条指令在流水线中重叠执行时，若指令 I_{k+1} 在指令 I_k 将计算结果写回目标寄存器之前取操作数，则将取出错误的操作数，从而导致指令执行结果错误。此类型的数据冒险对应于指令的真数据相关。

- WAR 冒险

指令 I_{k+1} 的目标寄存器与前一条指令 I_k 的源寄存器相同。当这两条指令在流水线中重叠执行时，若指令 I_{k+1} 在指令 I_k 读取源操作数之前完成了写回操作，则指令 I_k 将取出修改过的源操作数，从而导致指令执行结果错误。此类型的数据冒险对应于指令的名相关中的反相关。

- WAW 冒险

指令 I_{k+1} 与前一条指令 I_k 拥有相同的目标寄存器。当这两条指令在流水线中重叠执行时，若指令 I_{k+1} 在指令 I_k 写回之前写回，则目标寄存器最终的值将出现错误。此类型的数据冒险对应于指令的名相关中的输出相关。

? 思考题

RISC经典五级流水“IF-ID-EX-MEM-WB”是否存在WAR和WAW的数据冒险？为什么？

数据冒险可通过[流水线暂停](#)、[数据前递](#)和[乱序执行](#)等技术解决。

2.3 控制冒险

指令出现控制相关时，若分支指令尚未执行完成，则无法确定接下来需要执行哪条指令，从而出现流水线进入停顿状态的现象。此种情形称为流水线的控制冒险。

📌 举例说明

在流水线处理器中，分支指令完成执行阶段之前，无法确定下一条应该执行什么指令，如图3-2所示。

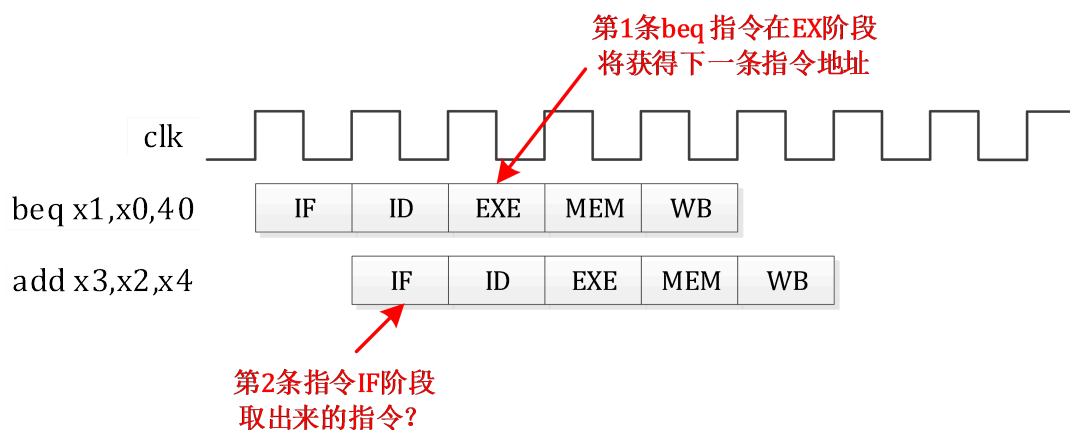


图3-2 控制冒险示意图

控制冒险可通过[流水线暂停](#)或引入[分支预测](#)机制等方法解决。

数据冒险的处理

1. 数据冒险的检测

要想解决流水线处理器中的数据冒险问题，首先需要在指令流中检测出数据冒险。

下面以图4-1为例介绍RAW冒险三种情形的检测方法。

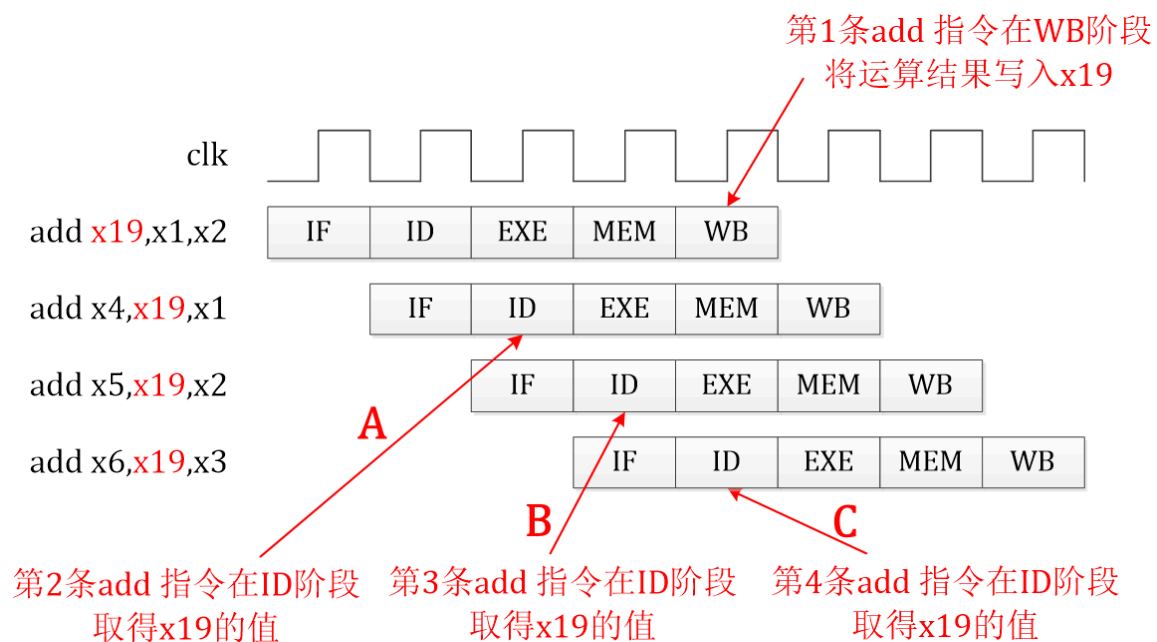


图4-1 RAW数据冒险的三种情形

1.1 RAW情形A检测

相邻指令发生RAW冒险 —— 图4-1中，第2条指令在译码阶段访问的寄存器与第1条指令在执行阶段将要写回的寄存器冲突。

流水线CPU中，与情形A有关的数据通路如图4-2所示。

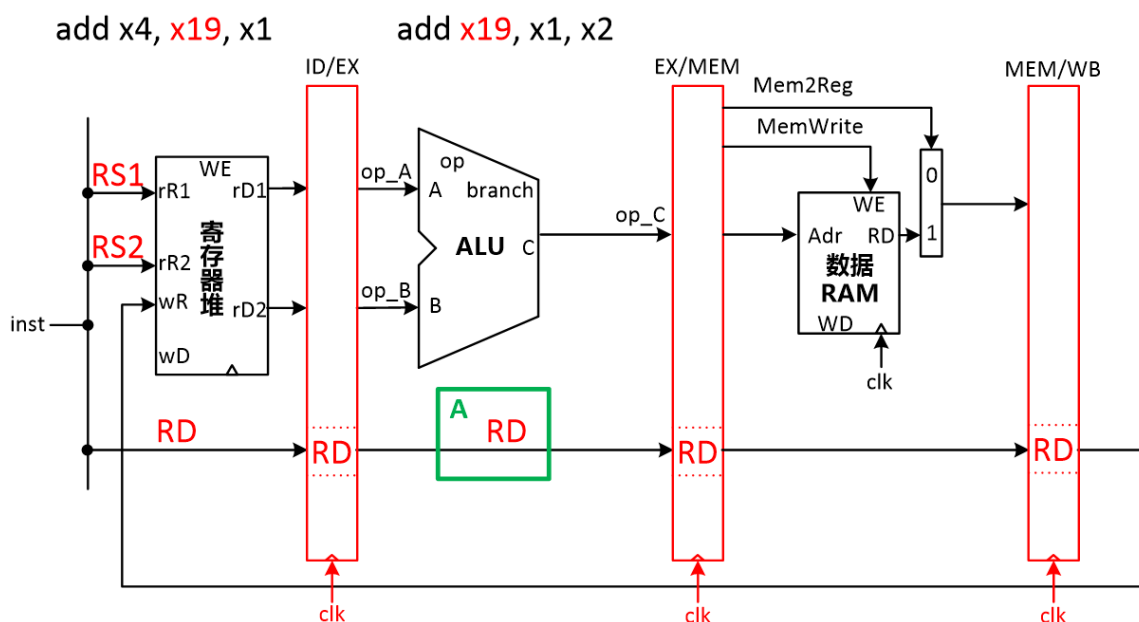


图4-2 RAW情形A有关的流水线数据通路

分析图4-2所示的数据通路，不难发现RAW情形A的判断方法/检测逻辑如下：

$$(\text{REG}_{\text{ID/EX.RD}} == \text{ID.RS1}) \parallel (\text{REG}_{\text{ID/EX.RD}} == \text{ID.RS2})$$

其中，REG_{ID/EX}表示译码阶段和执行阶段之间的流水线寄存器，下同；REG_{ID/EX}.RD表示ID/EX流水线寄存器的RD字段，观察图4-2可知该字段暂存的是从ID级传递而来的目标寄存器的寄存器号。

显然，对于图4-1中的情形A，有 $REG_{ID/EX}.RD == ID.RS1 == 5'd19$ 。

1.2 RAW情形B检测

间隔1条指令发生RAW冒险——图4-1中，第3条指令在译码阶段访问的寄存器与第1条指令在访存阶段将要写回的寄存器冲突。

流水线CPU中，与情形B有关的数据通路如图4-3所示。

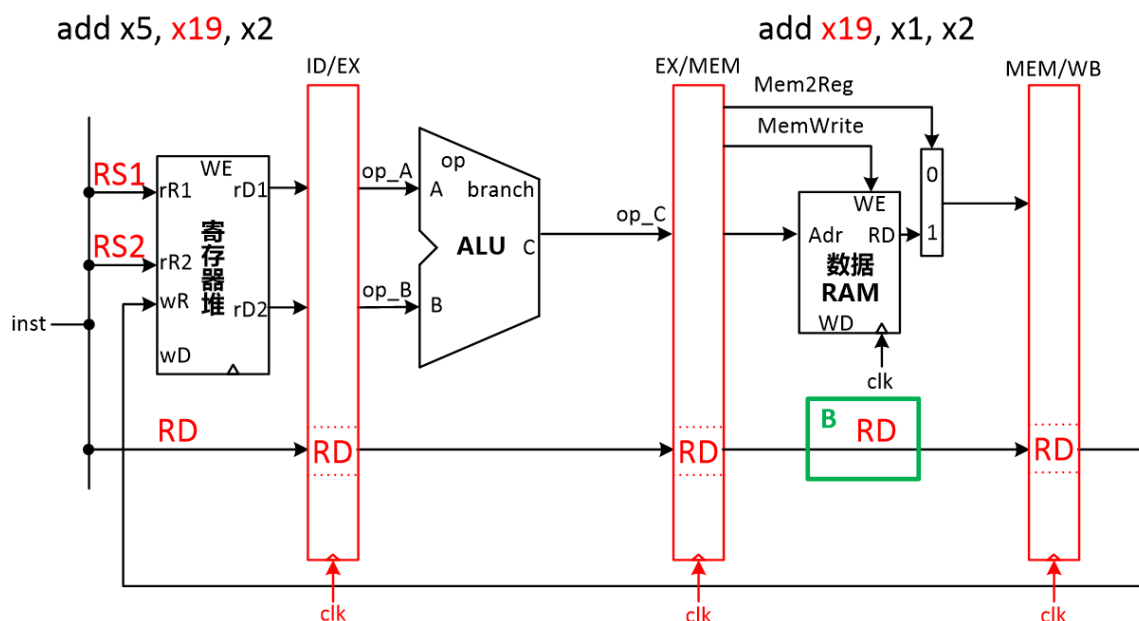


图4-3 RAW情形B有关的流水线数据通路

分析图4-3所示的数据通路，不难发现RAW情形B的判断方法/检测逻辑如下：

$$(\text{REG}_{\text{EX/MEM}}.\text{RD} == \text{ID}.\text{RS1}) \parallel (\text{REG}_{\text{EX/MEM}}.\text{RD} == \text{ID}.\text{RS2})$$

显然，对于图4-1中的情形B，有 $\text{REG}_{\text{EX/MEM}}.\text{RD} == \text{ID}.\text{RS1} == 5'd19$ 。

1.3 RAW情形C检测

间隔2条指令发生RAW冒险 —— 图4-1中，第4条指令在译码阶段访问的寄存器与第1条指令在写回阶段将要写回的寄存器冲突。

流水线CPU中，与情形C有关的数据通路如图4-4所示。

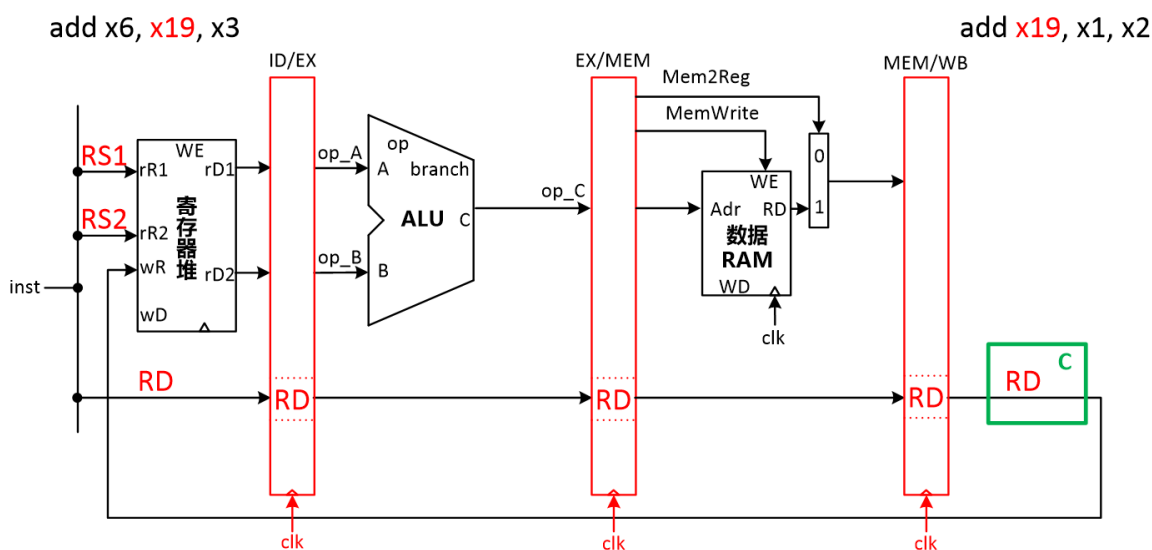


图4-4 RAW情形C有关的流水线数据通路

分析图4-4所示的数据通路，不难发现RAW情形C的判断方法/检测逻辑如下：

$$(\text{REG}_{\text{MEM/WB}}.\text{RD} == \text{ID}.\text{RS1}) \parallel (\text{REG}_{\text{MEM/WB}}.\text{RD} == \text{ID}.\text{RS2})$$

显然，对于图4-1中的情形C，有 $\text{REG}_{\text{MEM/WB}}.\text{RD} == \text{ID}.\text{RS1} == 5'h19$ 。

参考实现 ✨

对于RAW情形C，给出检测逻辑的参考实现代码：

```
1 wire rs1_id_wb_hazard = (wb_rd == id_rs1) & wb_we & id_rf1 & (wb_rd != 5'h0);
2 wire rs2_id_wb_hazard = (wb_rd == id_rs2) & wb_we & id_rf2 & (wb_rd != 5'h0);
```

图4-5 检测RAW冒险情形C的RTL参考实现

其中，`wb_rd` 表示WB阶段的目标寄存器的寄存器号；`wb_we` 表示WB阶段的写使能信号；`id_rs1` 表示ID阶段的RS1寄存器号；`id_rf1` 表示ID阶段的RS1寄存器的读标志信号，该信号用于标识ID阶段的RS1是否被读取。

特别注意的是，此处增加 `id_rf1` 和 `id_rf2` 信号是为了避免I型、U型和J型指令执行时出现RAW冒险的误判。此外，零号寄存器不可写，因而不存在RAW冒险，故而需要排除之。

请同学们参考上述实现代码，自行完成RAW三种情形的检测逻辑。

添加了数据冒险检测逻辑的流水线CPU数据通路形如图4-6所示。

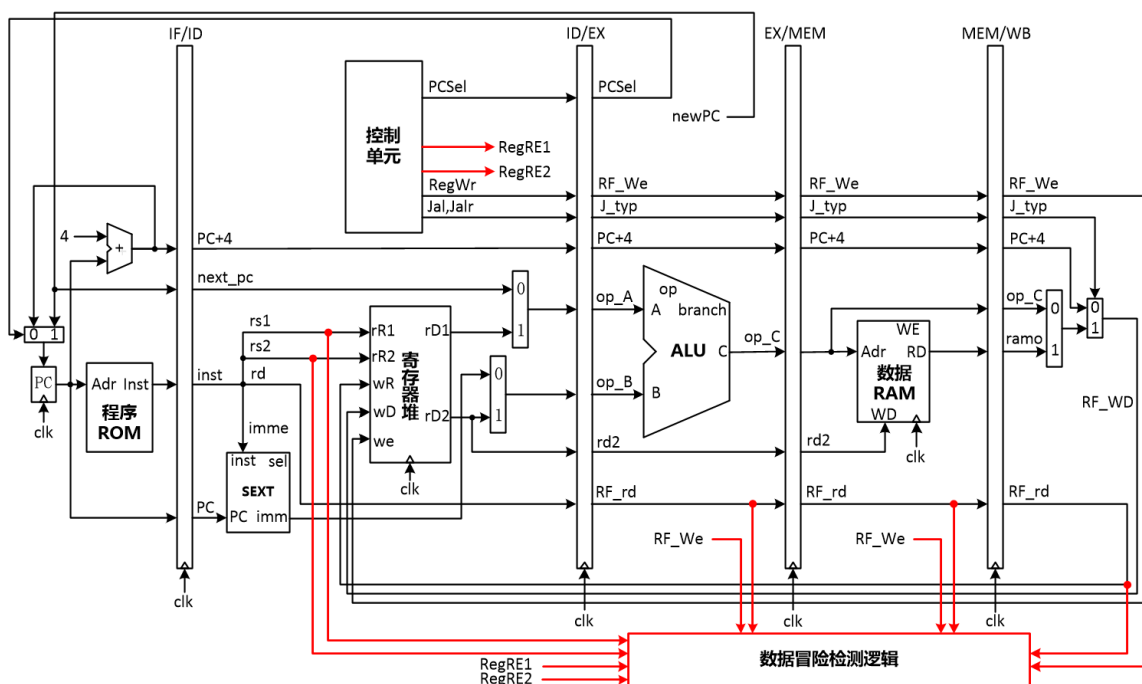


图4-6 具有数据冒险检测功能的流水线CPU数据通路示例

2. 数据冒险解决方法

2.1 流水线暂停

流水线发生数据冒险时，最朴素的解决方法就是暂停后续的指令，等待前面的指令执行完后，再重新让后续的指令执行。

举栗说明

对于如图4-1所示的RAW情形A，可让第二条指令暂停3个时钟周期，等待第一条指令写回后，再继续执行，如图4-7所示。

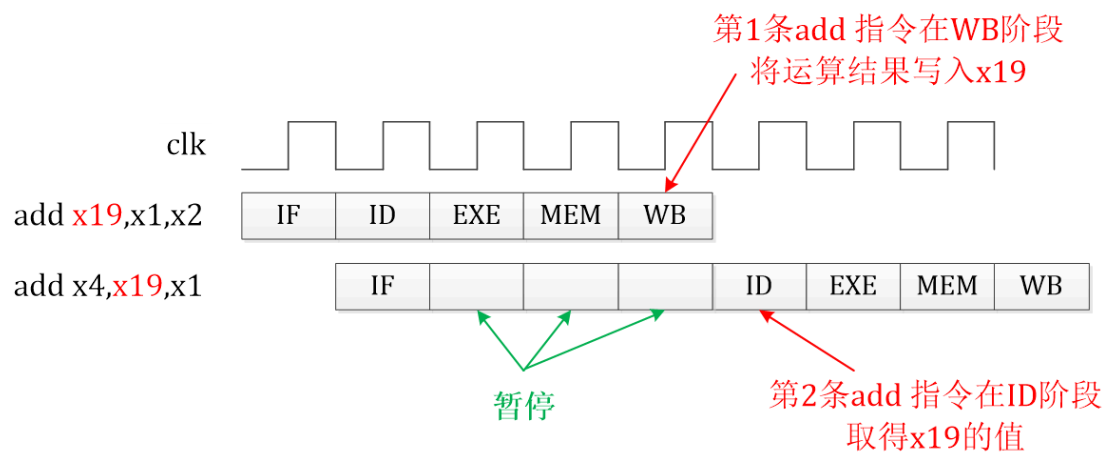


图4-7 流水线暂停

流水线暂停是功能最强大的冒险解决方法，因此它将流水线中并行进行的操作串行化。暂停法除了可以解决数据冒险，还可用于实现访存指令和乘除法指令。

暂停法实现访存指令、乘法指令的思路

访存指令需要通过总线读写主存数据，需要多个时钟周期才能完成。乘法指令需要通过硬件乘法器完成运算，而硬件乘法也需要多个时钟周期才能完成。对于32bit的“IF-ID-EX-MEM-WB”的五级流水CPU，假设采用两位编码的Booth算法实现乘法器，乘法操作至少也需要16个时钟周期才能完成。因此，在不改变流水线深度的前提下，需要借助流水线暂停来延长执行阶段，才能实现乘法指令。

使用暂停法实现访存指令的思路是：在EX阶段或MEM阶段通过相应的控制信号（比如 `ram_rop`、`ram_wop` 等）判断指令是否是访存指令；如果是，则通过MREQ模块产生只有效一个时钟周期的访存请求信号（即 `daccess_ren` 或 `daccess_wen`），同时暂停流水线的IF阶段和ID阶段，直到访存完成（即 `daccess_valid` 或 `daccess_wresp` 有效），再令流水线恢复执行。

使用暂停法实现乘法指令的思路是：在EX阶段通过 `alu_op` 判断指令是否是乘法指令；如果是，则产生只有效一个时钟周期的乘法启动信号 `start`，同时暂停流水线的IF阶段和ID阶段，直到乘法器输出的 `busy` 信号从高电平变成低电平，此时乘法运算完成，再令流水线恢复执行。

要使流水线的执行暂停下来，只需暂停相关寄存器的更新即可，即：

- (1) 保持取指地址（即PC寄存器的值）不变；
- (2) 保持各个流水线寄存器（IF/ID、ID/EX、EX/MEM、MEM/WB）不变；

暂停寄存器更新的方法是：将寄存器的输入修改为寄存器自身的输出。

添加了流水线暂停机制的流水线CPU数据通路形如图4-8所示。

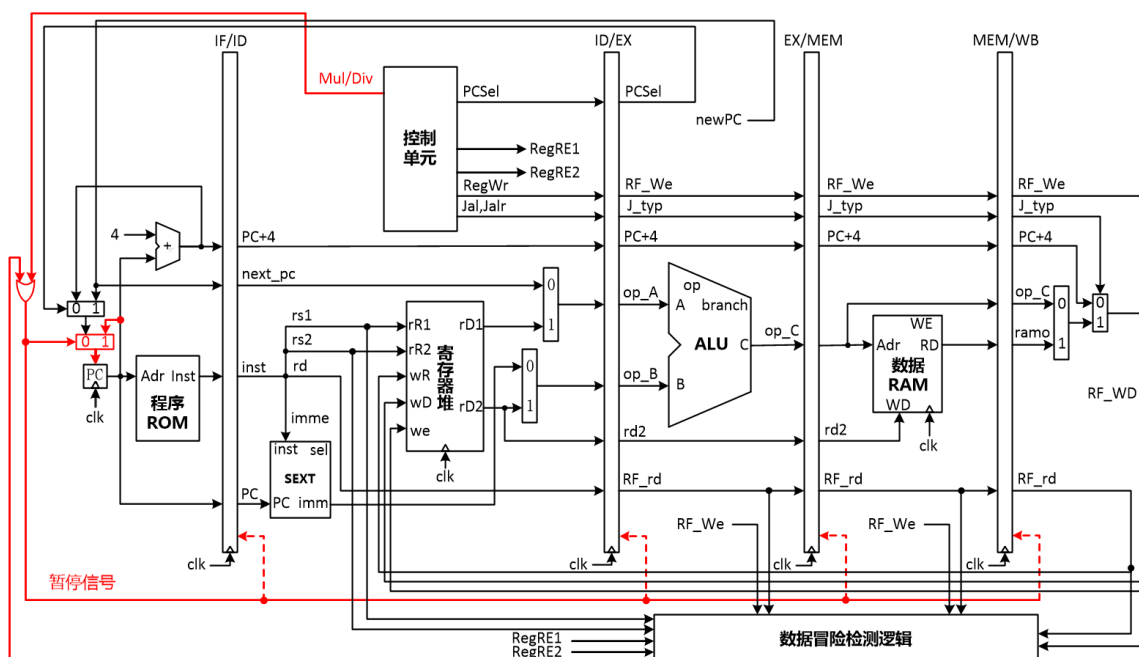


图4-8 具有暂停功能的流水线CPU数据通路示例

参考实现 ✨

图4-8中的虚线表示使用生成的暂停信号暂停流水线寄存器。限于空间，具体的暂停逻辑未画出。对于IF/ID寄存器，其暂停逻辑的参考RTL实现如下列代码所示。

```
1  always @(posedge clk or posedge rst) begin
2      if (rst) id_pc <= 32'h0;
3      else if (pipeline_stop) id_pc <= id_pc;
4      else id_pc <= if_pc;
5  end
6
7  always @(posedge clk or posedge rst) begin
8      if (rst) id_inst <= 32'h0;
9      else if (pipeline_stop) id_inst <= id_inst;
10     else id_inst <= if_inst;
11 end
12
13 .....
```

请同学们参考上述实现，自行完成相关寄存器的暂停操作。

2.2 数据前递

数据前递又称数据前推、数据旁路、数据转发，指的是将某个流水线阶段的中间结果向前传递到前面的阶段，且被前递的数据直接参与前面阶段中的操作。

举个栗子 🌰

对于图4-1所示的RAW冒险，可采用数据前递方法解决——将执行阶段、访存阶段或写回阶段的中间结果前递给译码阶段，并让前递而来的数据参与到指令译码时的源操作数选择，如图4-9所示。

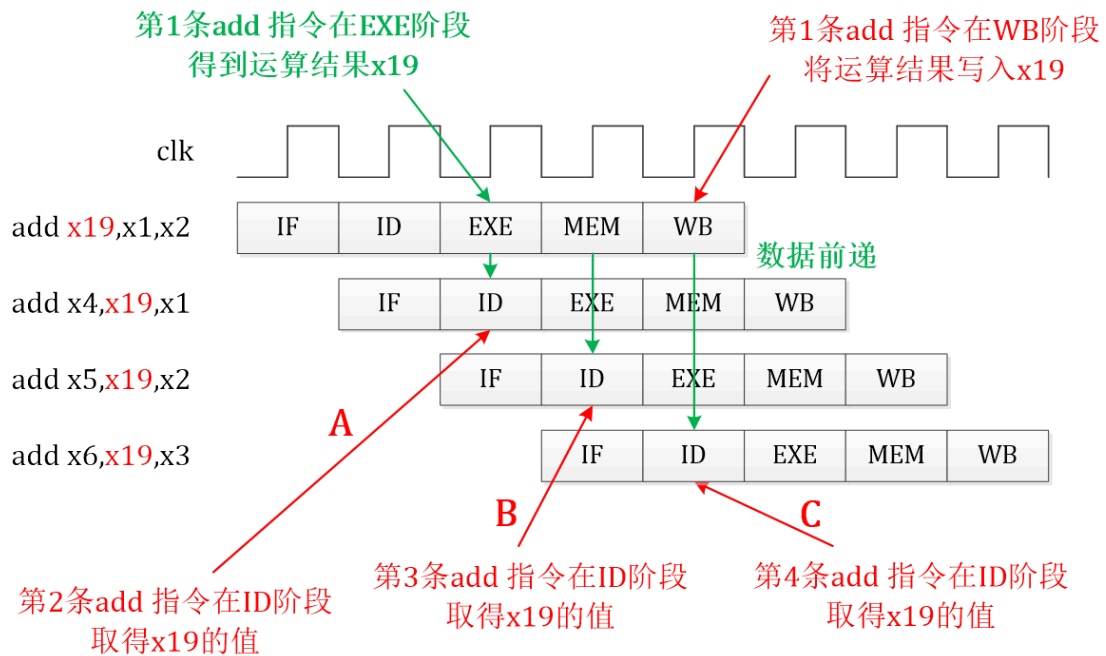


图4-9 数据前递示意图

图4-9中，从EXE指向ID、从MEM指向ID以及从WB指向ID的三条紫色箭头，分别对应图4-1中RAW冒险的三种情形。第一条指令在执行阶段即可产生计算结果（x19寄存器的新值），此时通过执行阶段到译码阶段的数据前递，可让第二条指令在译码时获得x19的新值，从而在不需暂停流水线的情况下解决了第一、二条指令之间的RAW冒险。

添加了数据前递机制的流水线CPU数据通路形如图4-10所示。

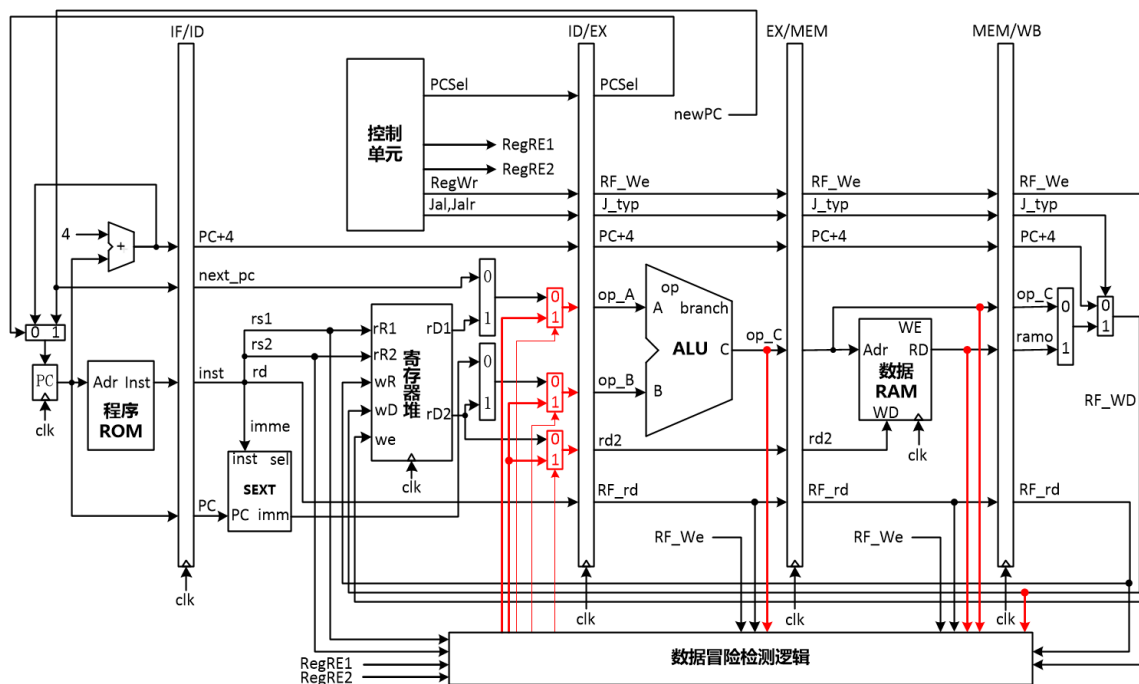


图4-10 增加数据前递机制的流水线CPU数据通路

参考实现 ✨

译码阶段被前递而来的数据，需要加入到源操作数的选择逻辑当中，相应的RTL参考实现如下列代码所示。

```

1  wire rs1_id_wb_hazard = (wb_rd == id_rs1) & wb_we & id_rf1;
2  wire rs2_id_wb_hazard = (wb_rd == id_rs2) & wb_we & id_rf2;
3
4  always @(posedge clk or posedge rst) begin
5      if (rst)          op_datA <= 32'h0;
6      else if (rs1_id_wb_hazard) op_datA <= forward_dat;
7      else               op_datA <= original_opA;
8  end
9
10 always @(posedge clk or posedge rst) begin
11     if (rst)          op_datB <= 32'h0;
12     else if (rs2_id_wb_hazard) op_datB <= forward_dat;
13     else               op_datB <= original_opB;
14 end

```

流水线暂停和数据前递并不冲突，有时甚至需要配合使用。

思考题 ✨

假设将图4-1中的第一条指令换成 `lw x19, 0(x1)`，考虑此时的RAW情形A，能否使用数据前递解决？

2.3 乱序执行

流水线发生数据冒险时，除了暂停和前递，还可以让后续的无相关关系的指令先执行，使得冒险问题自然而然地解决。

实现乱序执行的基本思路是，检测指令流中是否存在冒险；若存在冒险，则让后续的无相关指令先执行。乱序执行既可通过硬件动态实现，也可以通过编译器或汇编器静态实现。

典型的乱序执行算法包括记分牌算法和Tomasulo算法等，其硬件实现方法较为复杂，此处不作详细介绍。感兴趣的同学们可自行查阅相关资料。

对乱序执行感兴趣的同学，建议先尝试通过汇编器静态实现。

控制冒险的处理

按照RISC-V和LoongArch的指令集架构规范，控制冒险最简单的解决方法是 **默认预测所有指令都不发生跳转**。只有在EX阶段根据运算结果判断 **指令需要跳转时，再清除IF阶段和ID阶段，并使用正确的PC重新取指**。显然，在指令发生跳转时，会造成流水线性能的损失。

为了尽可能保持流水线的性能，一种思路是参考乱序执行的做法，在分支指令后方插入若干条无关指令，使得流水线的阶段始终处于工作状态，被插入的指令称为分支延迟槽指令；另一种思路是在分支结果产生之前，预测其可能的跳转方向和跳转目标地址，然后提前执行分支目标处的指令。

分支延迟槽一定程度上降低了汇编程序的可读性。此外，随着流水线深度的增加，延迟槽的实现也变得更加复杂，但其带来的性能提升较为有限。为了简化设计和节省芯片面积，RISC-V和LoongArch均已抛弃分支延迟槽技术，而是直接通过分支预测解决控制冒险问题。

1. 静态预测方法

最简单的静态分支预测方法是预测分支指令总是跳转，或预测总是不跳转。显然，使用这种方法对不同的程序进行分支预测时，预测准确率会有很大差异。一种改进的预测方法是若分支指令是往回跳的就预测跳转，否则预测不跳转，这种方法对单循环的预测效果较好。

静态分支预测的优点是实现简单、硬件开销较小、预测速度较快，但预测准确度相对较低，不能满足通用处理器对性能日益增长的需求。

在本课程中，实现默认预测不跳转的静态预测即可。感兴趣的同学可尝试实现动态预测方法。

2. 动态预测方法

动态分支预测根据分支指令的过去表现来预测其将来的行为。如果分支行为发生变化，那么分支预测的结果也相应地发生改变。因此，动态分支预测与静态分支预测相比具有更好的预测准确率和适应性。

2.1 基于BHT的分支预测

基于BHT（Branch History Table）的分支预测方法使用分支历史表来记录分支指令的历史行为。BHT的基础结构如图5-1所示。

Valid	Tag	分支历史(2bit)	分支目标地址
0	A ₀	H ₀	T ₀
1	A ₁	H ₁	T ₁
...	...	...	...
0	A _{k-1}	H _{k-1}	T _{k-1}

图5-1 基础BHT的结构

其中，Tag字段类似于Cache的Tag，是指令地址的一部分；分支历史由若干个2bit饱和计数器组成。预测时，取分支指令的地址去查BHT的Tag字段，然后根据BHT当前行的分支历史，对分支的跳转方向进行预测。如果BHT只记录各分支指令最近1次的分支历史，则只需使用1bit的饱和计数器。理论上，可使用任意位宽的饱和计数器来记录分支历史。有研究表明，2bit饱和计数器的预测性能与更高位宽的预测性能差不多。因此在实践中，通常使用2bit饱和计数器。

基于2bit饱和计数器进行分支预测的原理如图5-2所示。

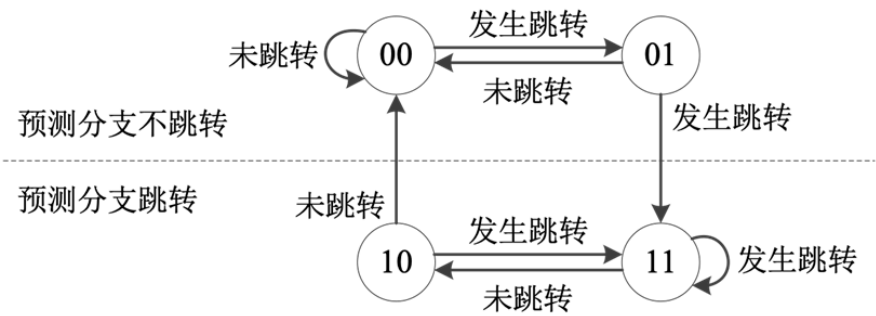


图5-2 基于2bit饱和计数器的分支预测

完整的分支预测过程包含预测和更新2个步骤。预测时，用分支指令的地址查BHT，获得相应的饱和计数器值。若饱和计数器的最高位为1，预测分支跳转，否则预测分支不跳转。当分支指令的实际跳转方向被确定时，不管预测是否正确，都根据图5-2对BHT中的饱和计数器进行更新，从而达到动态调整分支预测结果的目的。

2.2 基于全局历史的分支预测

基于BHT的分支预测方法忽视了分支指令之间的关联性。为此，基于全局历史的分支预测方法在BHT的基础上增加了GHR（Global History Register，全局历史寄存器）来将所有分支指令关联起来。

基于全局历史的分支预测方法使用一个k比特的GHR来记录所有最近k条分支指令的历史跳转方向，并使用PHT（Pattern History Table，模式历史表）来记录各分支指令的分支历史。其中，PHT的结构类似于BHT。

预测时，首先将分支指令的地址和GHR进行异或，再用异或操作的结果来查PHT，然后根据PHT当前行的分支历史和分支目标地址，对该分支指令的分支跳转方向和分支目标地址进行预测，如图5-3所示。

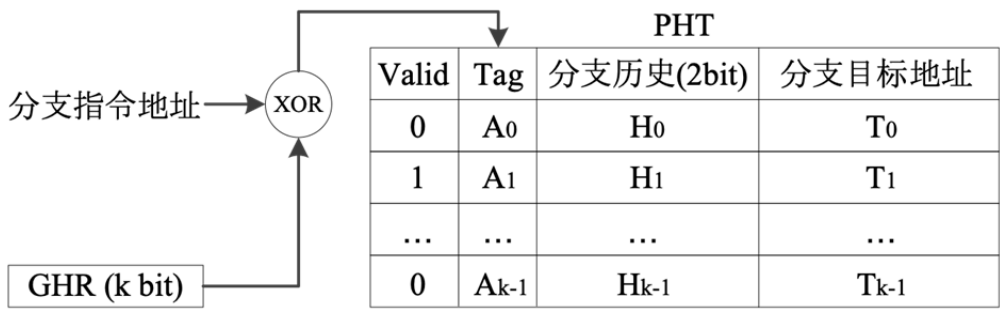


图5-3 基于全局历史的分支预测

当分支指令的实际跳转行为被确定时，GHR通过移位的方式进行更新——若指令跳转，则 $GHR = (GHR \ll 1) \mid 1$ ，否则 $GHR = (GHR \ll 1) \mid 0$ 。

2.3 基于局部历史的分支预测

基于全局历史的分支预测方法将所有分支指令都关联到一起。然而事实上，并非所有的分支指令都具有关联性。为此，基于局部历史的分支预测方法使用LHT（Local History Table，局部历史表）来代替全局历史预测中的GHR。

LHT一般具有64条记录，每条记录均包含Tag和局部转移历史2个字段。其中，Tag字段是分支指令地址的一部分，局部转移历史字段则是k比特的移位寄存器，其作用等同于GHR。

预测时，首先用分支指令的地址查LHT，得到分支指令的局部转移历史LHT[i]；然后将分支指令的地址和LHT[i]进行异或，再用异或操作的结果来查PHT；最后根据PHT当前行的分支历史和分支目标地址，对该分支指令的分支跳转方向和分支目标地址进行预测，如图5-4所示。

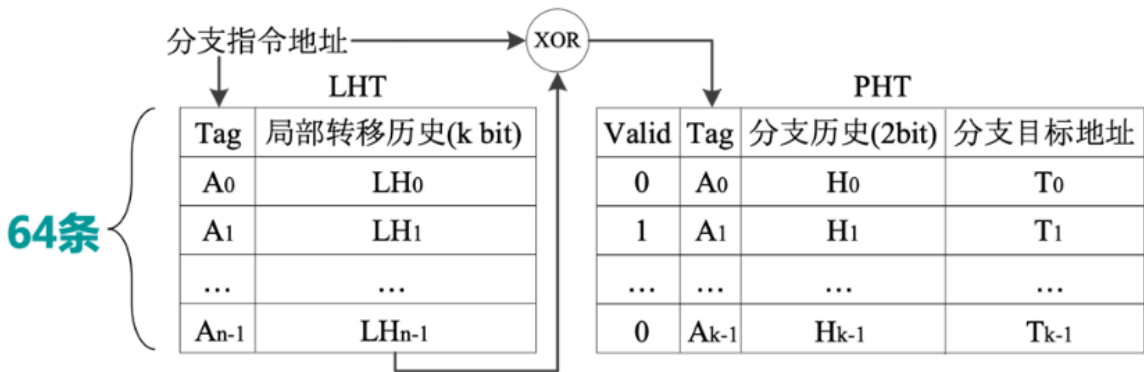


图5-4 基于局部历史的分支预测

当分支指令的实际跳转行为被确定时，LHT[i]和GHR一样，也通过移位的方式进行更新——若指令跳转，则 $LHT[i] = (LHT[i] \ll 1) \mid 1$ ，否则 $LHT[i] = (LHT[i] \ll 1) \mid 0$ 。

3. 分支预测的实现

CPU执行指令的过程中，检测到当前的指令是分支指令后，对分支结果进行预测，并根据预测结果继续后续的取指操作。需要注意的是，当分支预测失败时，需要清空分支指令之后的流水线中的指令，并修改PC以重新取指。

流水线的取指逻辑

前文在介绍理想流水线时，通过分析其时空图的特点，得出其取指逻辑如下列代码所示。

cpu_core.v

```
85 // 复位信号发生边沿变化时首次取指；上一条已取回，同时立即取下一条
86 assign ifetch_req = first_req | ifetch_valid;
87 assign ifetch_addr = pc;
```

在流水线能够处理数据冒险、控制冒险，以及访存指令和乘除法指令之后，我们需要对取指逻辑进行相应的调整，才能保证CPU能够正确运行。

考虑以下两种情况：

情况1：流水线发生暂停

引起流水线暂停的指令是访存指令和乘除法指令。访存指令需要读写主存，而取指操作也需要读取主存。由于系统采用哈佛结构，因此理论上取指和数据访存互不干扰。

在实际的计算机系统中，通常只有L1 Cache采用哈佛结构，而L2、L3 Cache以及主存、硬盘都是指令和数据不加区分进行存储的冯·诺依曼结构。我们的SoC系统最终也会采用冯·诺依曼结构的主存。因此，若CPU在指令访存期间继续取指，则 在ICache、DCache都发生缺失（或系统没有Cache）时，会出现取指请求和数据访问请求重叠的情况。这种情况会使得总线的设计变得复杂。

对于乘除法指令而言，流水线暂停期间不会出现访存请求重叠的情况。但是由于流水线的IF阶段和ID阶段都处于暂停状态，如果能让CPU继续取指，则需要增加指令缓存器，把流水线暂停期间取到的指令先缓存起来。等到流水线暂停解除之后，就能够直接执行这些指令。

简言之，如果是访存指令引起流水线暂停，则此时取指需要总线具备多个访存请求同时处理的能力；而如果是乘除法指令引起流水线暂停，则需要在CPU中增设指令缓存器。这些手段能够尽量保证流水线CPU的性能，但缺点是实现复杂度较高。最简单的处理方式是，在流水线暂停期间暂停取指。

实现时，我们需要 **在ID阶段检测到访存指令或乘除法指令时，就暂停取指；此外，在访存结束或乘除法运算完成后，恢复取指**，如下列代码所示。

cpu_core.v

```
wire pause_ifetch = (ldst_suspend | id_is_ld_st | ex_is_ld_st) & !ldst_done |
                    (mul_div_suspend | id_is_mul_div) & !mul_div_done;
wire resume_ifetch = ldst_done | mul_div_done;
```

```
assign ifetch_req = !pause_ifetch & (first_req | // 复位后首次取指
                                     ifetch_valid | // 上一条已取回，同时立即取下一
条
                                     resume_ifetch); // 数据访存或乘除运算结束，继续
取指
assign ifetch_addr = pc;
```

其中，ldst_suspend 是访存指令执行时产生的流水线暂停信号；id_is_ld_st、ex_is_ld_st 分别是ID阶段、EX阶段执行的指令是否是访存指令的标志位信号；ldst_done 是访存操作完成的标志位信号。其他依此类推。

pause_ifetch 信号从访存指令或乘除法指令进入ID阶段开始有效，直到访存操作及乘除法运算结束时无效。在 pause_ifetch 有效期间，CPU暂停取指。

当访存操作结束（即 ldst_done 有效）或乘除法运算结束（即 mul_div_done 有效），resume_ifetch 信号有效，此时CPU恢复取指。

情况2：分支预测失败

流水线默认预测指令不跳转，从而不停地顺序取指执行。当静态分支预测失败时，流水线需要清除IF阶段、ID阶段并 使用EX阶段计算出的跳转目标地址重新取指。因此，我们的流水线CPU需要有一个指令发生跳转的标志位信号 —— 当这个标志位信号有效时，CPU需要发出取指请求，且取指的地址由EX阶段计算得出。

综合情况1，我们可以得到流水线CPU的取指逻辑如下列代码所示。

```
cpu_core.v

wire pause_ifetch = (ldst_suspend | id_is_ld_st | ex_is_ld_st) & !ldst_done |
                    (mul_div_suspend | id_is_mul_div) & !mul_div_done;
wire resume_ifetch = ldst_done | mul_div_done;

assign ifetch_req = !pause_ifetch & (first_req | // 复位后首次取指
                                     ifetch_valid | // 上一条已取回，同时立即取下一
条
                                     ex_bj_f | // 静态分支预测错误，立即用正确
的地址取指
                                     resume_ifetch); // 数据访存或乘除运算结束，继续
取指
assign ifetch_addr = ex_bj_f ? ex_bj_target : pc;
```

其中，ex_bj_f 是EX阶段产生的指令跳转的标志位信号，而 ex_bj_target 则是EX阶段计算得出的跳转目标地址。

实验步骤

(1) 把单周期CPU的工程及代码复制到另一个文件夹，然后划分单周期CPU的数据通路，并在模板工程上实现理想流水线CPU；

(2) 使用无指令相关性、无访存、无乘除法的简单程序，对理想流水线CPU进行 仿真测试；

(3) 为理想流水线CPU增加冒险检测逻辑，并为分支跳转指令修改取指逻辑，使用 默认预测不跳转 的静态分支预测解决流水线的控制冒险；

(4) 实现暂停法解决流水线冒险，并为访存指令和乘法指令修改取指逻辑；

(5) 修改流水线数据通路，实现数据前递法解决数据冒险；

(6) 使用Trace测试框架，对具有冒险检测和处理功能的流水线CPU进行功能验证，使其通过Basic Trace测试；

如果需要在Vivado中运行功能仿真

使用bin2coe.py脚本，可将Trace测试框架中的 `cdp-tests / bin` 文件夹下的.bin文件转换成.coe文件。

参考实验1-实验原理-功能部件设计-3.2 Block Memory的初始化-图3-8，把.coe文件导入工程的IROM IP核当中，再运行功能仿真即可。

如果xx.bin文件对应在 `cdp-tests / asm` 目录的xx.dump程序包含有数据段，则需要把.coe文件同时导入IROM和DRAM的IP核。

(7) 与组员讨论协作，把流水线CPU集成到SoC，并使用AXI Trace调试和验证流水线SoC；

如果需要在Vivado中运行功能仿真

对于支持AXI总线的SoC，只需把.coe文件导入 `bram_axi` 的IP核即可运行功能仿真。

(8) 与组员讨论协作，在SoC上完成CoreMark或LLAMA2推理程序的下板测试；

(9) 尝试优化性能（比如提高频率、优化访存、优化运算等），减少CoreMark的运行时间，或提升LLAMA2的推理速率。

CoreMark下板运行效果示例

SoC下板运行CoreMark的效果如图7-1所示。

```
CoreMark 1.0
2K performance run parameters for coremark.
CoreMark Size      : 666
Total ticks        : 897896501
Total time (secs)  : 11
Iterations/Sec     : 63
Iterations         : 700
Compiler version   : GCC10.2.0
Compiler flags     : -O2 -funroll-loops -fpeel-loops -fgcse-sm -fgcse-las
Memory location    : STATIC
seedcrc           : 0xe9f5
[0]crclist        : 0xe714
[0]crcmatrix      : 0x1fd7
[0]crcstate       : 0x8e3a
[0]crcfinal       : 0x65c5
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 62.3679
CoreMark/MHz : 0.7795
FINISH
```

图7-1 CoreMark下板运行效果示意图

若CoreMark下板运行出现问题，可将C_TEST中的测试程序逐一下板测试，并结合其代码进行调试。

注意：提升CPU时钟频率后，需相应修改C程序的频率参数

提高CPU时钟频率的方法见[常见问题 - 1.7 如何提高CPU主频？](#)

模板工程的CPU时钟默认为50MHz。改变频率后，需要相应地修改C程序中的频率参数，否则程序计时不准确。

具体地，我们需要修改CoreMark源文件 `core_portme.c` 第36行的 `MHZ` 参数，使其等于实际的CPU频率：

```
c_test / 4_coremark / src / coremark / core_portme.c

36 #define MHZ 50 // CPU Clock Frequency
37 #define CLOCKS_PER_SEC (1000000 * MHZ)
```